

SBA301 – INTEGRATE SINGLE PAGE APPLICATION WITH SPRING BOOT

STUDENT LAB GUIDE

LAB 04.1

SPRING BOOT RESTFUL WEB SERVICE WITH MONGODB NOSQL

Upgrade path from Lab 04: JPA / SQL Database → Spring Data MongoDB / Document Database

REST Controller → Service → MongoRepository → MongoDB → JSON Response

Thông tin	Giá trị
Mục tiêu	Củng cố REST Controller và học phương pháp thay persistence source từ relational database sang MongoDB NoSQL
Baseline	JDK 21 • Spring Boot 3.x • Maven • Spring Web • Spring Data MongoDB • Validation
Công cụ	IntelliJ IDEA / VS Code • MongoDB Community Server • MongoDB Compass • Postman
Case study	Product – Category – Review
Thời lượng gợi ý	2 buổi thực hành + tự học / evidence completion

Tư duy trung tâm: Đây không phải là bài “đổi JpaRepository thành MongoRepository”. Sinh viên phải hiểu vì sao REST contract có thể giữ ổn định trong khi persistence model, relationship strategy, query semantics và testing strategy thay đổi.

Version: October 2026

0. MỤC TIÊU, PHẠM VI VÀ CÁCH SỬ DỤNG LAB 04.1

0.1. Lab 04.1 là gì?

Lab 04.1 là phiên bản nâng cấp/biến thể của Lab 04 JPA. Bài lab giữ cùng tư duy RESTful API và kiến trúc 3-layer, nhưng thay nguồn dữ liệu relational bằng MongoDB. Mục tiêu không phải phủ định JPA; mục tiêu là giúp sinh viên nhìn thấy ranh giới giữa application/API design và persistence technology.

0.2. Sau Lab 04.1, sinh viên phải làm được

- Cài đặt và xác minh MongoDB Community Server trên Windows.
- Cài đặt MongoDB Compass và kết nối tới MongoDB local.
- Cấu hình Spring Boot kết nối MongoDB bằng URI.
- Chuyển domain model từ JPA entity sang MongoDB document.
- Thay JpaRepository bằng MongoRepository mà không phá vỡ REST contract.
- Thiết kế embedded document và reference theo access pattern thay vì suy nghĩ theo foreign key/join.
- Triển khai CRUD, validation, exception handling, paging, sorting, filtering và indexing.
- Kiểm chứng dữ liệu BSON/document bằng Compass và HTTP contract bằng Postman.
- So sánh được SQL/JPA và MongoDB ở mức architecture, modeling, query, transaction và testing.
- Giải thích được những phần nào của Controller/Service giữ nguyên và những phần nào phải thay đổi khi đổi database.

0.3. Điều kiện đầu vào

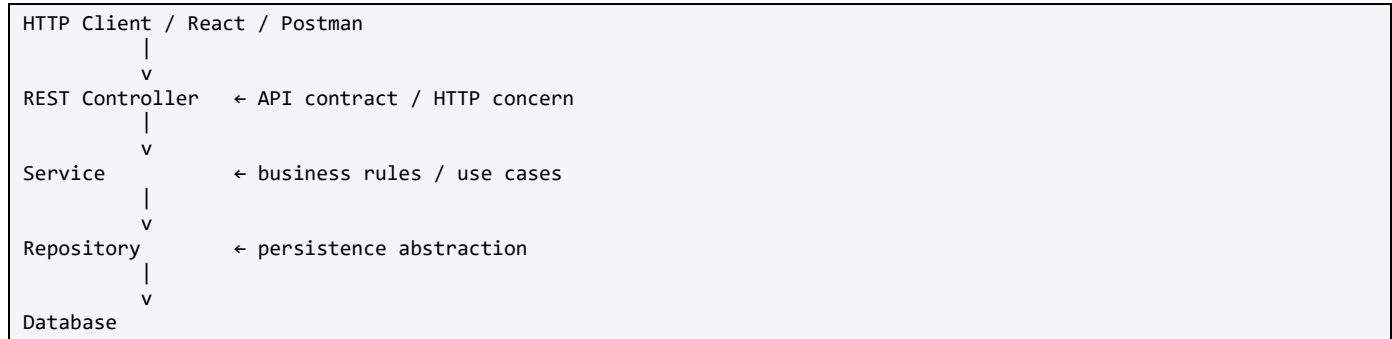
- Đã hoàn thành hoặc đọc Lab 04 JPA.
- Hiểu @RestController, @RequestMapping, DTO, @Valid và ResponseEntity.
- Hiểu Entity → Repository → Service → Controller.
- Biết CRUD và status code cơ bản.
- Có JDK 21, IDE, Maven và Postman.

0.4. Phạm vi cố ý loại trừ

- Không triển khai MongoDB cluster/sharding ở mức production.
- Không đi sâu replica set transaction nhiều document.
- Không dùng Spring Reactive/WebFlux trong core lab.
- Không yêu cầu migrate dữ liệu production từ SQL Server sang MongoDB.
- Không dùng MongoDB như “SQL không cần schema”; document modeling vẫn phải có quyết định và bằng chứng.

I. BƯỚC TRANH KIẾN TRÚC: LAB 04 JPA → LAB 04.1 MONGODB

1.1. Kiến trúc nào giữ nguyên?



Controller và Service không nên “biết quá nhiều” về chi tiết database. Nếu boundaries được thiết kế tốt, thay persistence technology sẽ tập trung chủ yếu ở model/repository/configuration và một phần service query logic.

1.2. Bảng nâng cấp từ Lab 04

Layer / concern	Lab 04 – JPA / SQL	Lab 04.1 – MongoDB	Mức thay đổi
REST endpoint	GET/POST/PUT/DELETE /api/products	Giữ cùng contract	Rất ít
Controller	@RestController, DTO, ResponseEntity	Giữ gần như nguyên	Rất ít
Service	Business rules + transaction	Business rules; query logic có thể đổi	Ít-vừa
Model	@Entity, @Table, @Column	@Document, fields, embedded subdocuments	Lớn
Identifier	Long + @GeneratedValue	String/ObjectId; MongoDB sinh _id	Lớn
Repository	JpaRepository<Product, Long>	MongoRepository<Product, String>	Lớn
Relationship	FK + JOIN + JPA association	Embed / manual reference / @DocumentReference	Rất lớn
Schema	Tables/columns/constraints	Collections/documents; flexible nhưng có model	Lớn
Query	JPQL/derived/native SQL	Derived/Mongo query/aggregation	Vừa-lớn
Index	RDBMS indexes	MongoDB indexes / @Indexed	Vừa
Persistence test	@DataJpaTest	Mongo integration/slice/Testcontainers	Lớn
Inspection	SQL Server/H2 console	MongoDB Compass	Lớn

1.3. Nguyên tắc “API stable – persistence replaceable”

Architecture rule: Frontend không nên phải đổi chỉ vì backend chuyển từ SQL/JPA sang MongoDB, miễn API contract vẫn giữ method, URL, request DTO, response DTO và error format tương thích.

1.4. Điều gì KHÔNG được làm khi chuyển MongoDB?

- Không cố tìm “annotation Mongo tương đương 1:1” cho mọi @OneToMany/@ManyToMany.
- Không copy nguyên relational ERD rồi tạo mỗi table thành một collection.
- Không embed một mảng có thể tăng vô hạn vào parent document.
- Không dùng reference cho mọi thứ chỉ vì quen foreign key.
- Không expose Mongo document trực tiếp như API contract nếu domain có khả năng thay đổi.

II. CÀI ĐẶT VÀ CẤU HÌNH MONGODB TRÊN WINDOWS

2.1. Thành phần sẽ cài

Thành phần	Vai trò	Bắt buộc?
MongoDB Community Server	Database server chạy tiến trình mongod	Có
MongoDB Compass	GUI để xem database/collection/document/index/query	Khuyến nghị mạnh
mongosh	CLI shell cho MongoDB	Tùy chọn nhưng hữu ích
Postman	Test REST API	Có cho lab

Current note: MongoDB documentation hiện hướng dẫn Community Edition 8.0 trên Windows bằng MSI. MongoDB Shell (mongosh) không đi kèm MongoDB Server và cần cài riêng nếu muốn dùng CLI. Trong khóa học, sinh viên có thể chỉ cần Server + Compass + Spring Boot.

2.2. Bước 1 – Download MongoDB Community Server

1. Truy cập MongoDB Download Center.
2. Chọn MongoDB Community Server.
3. Chọn Platform = Windows.
4. Chọn Package = MSI.
5. Tải installer về máy.

Không cần ép sinh viên dùng một patch version cụ thể. Giảng viên có thể khóa một major/minor version cho cả lớp để dễ hỗ trợ. Với máy lab, nên dùng cùng version đã kiểm thử trước.

2.3. Bước 2 – Chạy MSI installer

6. Mở file .msi bằng quyền phù hợp.
7. Chọn Complete installation nếu không có yêu cầu đặc biệt.
8. Giữ lựa chọn cài MongoDB như Windows Service.
9. Service name mặc định có thể là MongoDB.
10. Giữ data/log directory do installer đề xuất hoặc ghi lại đường dẫn nếu thay đổi.
11. Chọn Install MongoDB Compass nếu installer cung cấp lựa chọn này.
12. Hoàn tất installation.

2.4. Bước 3 – Kiểm tra Windows Service

```
Windows Search → Services
Tìm service: MongoDB
Status mong muốn: Running
Startup Type gợi ý: Automatic
```

Nếu service chưa chạy, Start service. Nếu dùng command line có thể kiểm tra bằng PowerShell/CMD với quyền phù hợp.

```
sc query MongoDB
```

2.5. Bước 4 – Kiểm tra default local connection

MongoDB local mặc định thường chỉ bind localhost. Connection string dùng cho Lab:

```
mongodb://localhost:27017
```

Security note: Không mở bindIp ra public network cho bài lab. Nếu phải cho máy khác truy cập, cần học authentication/network hardening trước. Localhost là cấu hình an toàn và đủ cho lớp học.

2.6. Bước 5 – Cài/khởi động MongoDB Compass

13. Khởi động MongoDB Compass.
14. Tại New Connection, nhập mongodb://localhost:27017.
15. Connect.

16. Nếu thành công, Compass hiển thị server local và các database hệ thống.
 17. Chưa thấy sba301_lab41 là bình thường; database sẽ xuất hiện sau khi application tạo dữ liệu.

2.7. Bước 6 – mongosh (tùy chọn)

MongoDB Server không nhất thiết cài kèm mongosh. Nếu giảng viên muốn sinh viên luyện CLI, cài mongosh riêng theo MongoDB documentation, sau đó kiểm tra:

```
mongosh --version
mongosh "mongodb://localhost:27017"
```

2.8. Troubleshooting cài đặt

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra/xử lý
Compass cannot connect	MongoDB service chưa chạy	Services → MongoDB → Start; kiểm tra port 27017
Connection refused	Sai host/port hoặc service stop	Dùng mongodb://localhost:27017; kiểm tra netstat/service
mongosh not recognized	mongosh chưa cài/PATH chưa có	Cài mongosh riêng hoặc dùng Compass
Port 27017 occupied	Process khác dùng port	Kiểm tra port; không đổi port tùy tiện nếu lớp dùng config chung
Remote connection fails	MongoDB bind localhost	Lab chỉ dùng local; không mở public nếu chưa cấu hình security

III. TẠO SPRING BOOT PROJECT CHO LAB 04.1

3.1. Dependency map

Dependency	Mục đích
Spring Web	REST Controller / HTTP
Spring Data MongoDB	MongoTemplate + Mongo repositories + mapping
Validation	@Valid và Bean Validation
Spring Boot Test	JUnit/Mockito/MockMvc
Springdoc OpenAPI (nếu course project đã dùng)	Swagger UI / API documentation

3.2. Maven dependencies cốt lõi

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

3.3. Cấu trúc package

```
com.example.lab41
├── Lab41Application.java
├── controller/
├── dto/
├── exception/
├── model/
├── repository/
├── service/
└── config/
```

3.4. application.properties

```
spring.application.name=lab41-mongodb
spring.data.mongodb.uri=mongodb://localhost:27017/sba301_lab41

# Useful during learning; do not enable noisy logs in production without reason
logging.level.org.springframework.data.mongodb.core.MongoTemplate=DEBUG
```

Chỉ cần URI là đủ cho local lab. Database sba301_lab41 sẽ được tạo khi MongoDB thực sự ghi document đầu tiên.

3.5. Kiểm tra startup

18. Run Spring Boot application.
19. Không được có lỗi connection refused tới MongoDB.
20. Mở Compass và giữ connection local.
21. Chưa cần collection products cho đến khi seed hoặc POST tạo dữ liệu.

IV. DOCUMENT MODEL: TỪ @ENTITY SANG @DOCUMENT

4.1. Product trong Lab 04 JPA

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private BigDecimal price;
}
```

4.2. Product trong Lab 04.1 MongoDB

```
@Document(collection = "products")
public class Product {
    @Id
    private String id;

    private String name;
    private BigDecimal price;
    private Integer stock;
    private String categoryId;
    private List<Review> reviews = new ArrayList<>();
}
```

4.3. Mapping comparison

Relational/JPA	MongoDB	Giải thích
@Entity	@Document	Object persistent dưới dạng document
@Table	collection="products"	Collection thay table
Long ID + @GeneratedValue	String/ObjectId mapped vào _id	MongoDB có thể sinh identifier
@Column	field mặc định / @Field khi cần	Không cần annotation cho mọi field
FK category_id	categoryId hoặc @DocumentReference	Không có relational FK enforcement
Child table Review	Embedded List<Review> hoặc collection riêng	Quyết định theo access pattern

4.4. DTO vẫn nên giữ

```
public record ProductRequest(
    @NotBlank String name,
    @NotNull @PositiveOrZero BigDecimal price,
    @NotNull @PositiveOrZero Integer stock,
    String categoryId
) {}

public record ProductResponse(
    String id,
    String name,
    BigDecimal price,
    Integer stock,
    String categoryId,
    List<ReviewResponse> reviews
) {}
```

Boundary rule: MongoDB flexible schema không phải lý do để bỏ DTO/validation. API contract vẫn phải ổn định và có validation boundary.

V. EMBED HAY REFERENCE? – TRỌNG TÂM NOSQL MODELING

5.1. Vì sao không map relationship theo kiểu JPA 1:1

Trong relational database, normalization và foreign key thường dẫn cách thiết kế. Trong MongoDB, câu hỏi thiết kế quan trọng hơn là dữ liệu được đọc/ghi cùng nhau như thế nào, kích thước có tăng không giới hạn không, và object có lifecycle độc lập không.

5.2. Embedded Review

```
public class Review {
    private String userId;
    private int rating;
    private String comment;
    private Instant createdAt;
}

@Document(collection = "products")
public class Product {
    @Id
    private String id;
    private String name;
    private BigDecimal price;
    private List<Review> reviews = new ArrayList<>();
}
```

Với lab nhỏ, Review có thể embed nếu số review được kiểm soát và luôn hiển thị cùng Product. Trong hệ thống thật với hàng triệu review, embedding không giới hạn là thiết kế nguy hiểm; cần collection riêng/pagination.

5.3. Manual reference Category bằng ID

```
@Document(collection = "categories")
public class Category {
    @Id
    private String id;
    private String name;
}

// Product
private String categoryId;
```

Manual ID reference giúp sinh viên nhìn rõ rằng MongoDB không tự enforce foreign key. Service chịu trách nhiệm kiểm tra categoryId có hợp lệ khi business rule yêu cầu.

5.4. @DocumentReference awareness

```
@DocumentReference
private Category category;
```

Spring Data MongoDB hỗ trợ document references. Tuy nhiên referenced document không được cascade-save tự động theo cách sinh viên thường kỳ vọng từ JPA cascade. Lab core ưu tiên manual ID reference để làm rõ semantics.

5.5. Decision matrix

Câu hỏi thiết kế	Embed	Reference
Child luôn được đọc cùng parent?	Ưu tiên	Có thể
Child không có lifecycle độc lập?	Ưu tiên	Ít cần
Child nhỏ và bounded?	Ưu tiên	Có thể
Child được nhiều parent dùng chung?	Không nên	Ưu tiên
Child cập nhật độc lập thường xuyên?	Cần nhắc	Ưu tiên
Child có thể tăng rất lớn/không giới hạn?	Tránh	Ưu tiên collection riêng
Cần query child độc lập thường xuyên?	Không thuận lợi	Ưu tiên
Cần snapshot lịch sử ngay cả khi source thay đổi?	Có thể embed snapshot	Reference không giữ snapshot

5.6. Human Gate

22. Vì sao Review được embed nhưng Category dùng reference ID?
23. Nếu Product có 500.000 reviews, em có tiếp tục embed không? Vì sao?
24. MongoDB có foreign key constraint không? Nếu categoryId sai thì layer nào chịu trách nhiệm?
25. Khi nào duplication trong document model là chủ đích chứ không phải lỗi normalization?

VI. REPOSITORY: JPAREPOSITORY → MONGOREPOSITORY

6.1. Repository cơ bản

```
public interface ProductRepository
    extends MongoRepository<Product, String> {

    Page<Product> findByNameContainingIgnoreCase(
        String keyword,
        Pageable pageable
    );

    Page<Product> findByCategoryId(
        String categoryId,
        Pageable pageable
    );
}
```

6.2. Những gì vẫn quen thuộc

- save()
- findById()
- findAll()
- delete()
- existsById()
- Pageable / Page / Sort
- Derived query naming conventions

6.3. Những gì thay đổi

- Không còn JPQL.
- Không còn SQL JOIN semantics mặc định.
- Query document/nested fields dùng Mongo query semantics.
- Aggregation pipeline có vai trò khi query phức tạp.
- Index strategy phải dựa trên query pattern.

6.4. Query method examples

```
List<Product> findByPriceBetween(BigDecimal min, BigDecimal max);
List<Product> findByStockGreaterThan(Integer stock);
Page<Product> findByNameContainingIgnoreCaseAndCategoryId(
    String keyword,
    String categoryId,
    Pageable pageable
);
```

6.5. MongoTemplate awareness

MongoRepository đủ cho core lab. MongoTemplate nên được giới thiệu như công cụ query/mutation linh hoạt hơn khi repository method trở nên khó đọc. Không cần dùng MongoTemplate chỉ để chứng minh “NoSQL”.

VII. SERVICE LAYER: ÍT THAY ĐỔI NHƯNG TRÁCH NHIỆM MODELING TĂNG

7.1. ProductService

```
@Service
public class ProductService {
    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;

    public ProductService(ProductRepository productRepository,
                          CategoryRepository categoryRepository) {
        this.productRepository = productRepository;
        this.categoryRepository = categoryRepository;
    }

    public ProductResponse create(ProductRequest request) {
        validateCategory(request.categoryId());

        Product product = new Product();
        product.setName(request.name());
        product.setPrice(request.price());
        product.setStock(request.stock());
        product.setCategoryId(request.categoryId());

        return toResponse(productRepository.save(product));
    }

    private void validateCategory(String categoryId) {
        if (categoryId != null && !categoryRepository.existsById(categoryId)) {
            throw new ResourceNotFoundException("Category not found: " + categoryId);
        }
    }
}
```

7.2. Vì sao Service quan trọng hơn khi dùng manual reference?

Vì MongoDB không enforce relational foreign key, application service phải quyết định business consistency: category phải tồn tại hay không, khi xóa category có cho phép product giữ dangling reference không, có denormalize categoryName vào product không, v.v.

7.3. Transaction note

Single-document operations của MongoDB mang tính atomic ở document level. Multi-document transactions tồn tại nhưng thường yêu cầu replica set/sharded cluster và không nên là core lab local. Thiết kế document tốt thường cố gắng gom dữ liệu cần atomic update vào cùng document khi hợp lý.

VIII. REST CONTROLLER: GIỮ API CONTRACT ỔN ĐỊNH

8.1. Controller CRUD

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public ProductResponse getById(@PathVariable String id) {
        return service.findById(id);
    }

    @PostMapping
    public ResponseEntity<ProductResponse> create(
        @Valid @RequestBody ProductRequest request) {
        ProductResponse created = service.create(request);
        URI location = URI.create("/api/products/" + created.id());
        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public ProductResponse update(
        @PathVariable String id,
        @Valid @RequestBody ProductRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

8.2. Contract comparison

HTTP contract	Lab 04 JPA	Lab 04.1 MongoDB
POST /api/products	201 + Location	Giữ nguyên
GET /api/products/{id}	200/404	Giữ nguyên; id đổi Long → String
PUT /api/products/{id}	200/404/400	Giữ nguyên
DELETE /api/products/{id}	204/404	Giữ nguyên
Validation error	400 standardized error	Giữ nguyên
Frontend integration	Axios/Fetch	Không cần biết database đổi

8.3. Error handler vẫn giữ

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError("NOT_FOUND", ex.getMessage()));
    }
}
```

IX. PAGING, SORTING, FILTERING VÀ INDEX

9.1. Pageable

```
@GetMapping
public Page<ProductResponse> search(
    @RequestParam(defaultValue = "") String keyword,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "name") String sortBy) {

    Pageable pageable = PageRequest.of(
        page,
        Math.min(size, 50),
        Sort.by(sortBy).ascending()
    );

    return service.search(keyword, pageable);
}
```

9.2. Whitelist sort fields

Không truyền trực tiếp mọi giá trị sortBy vào production query. Lab nên whitelist name, price, stock để tránh API behavior khó đoán và query vào field không được index.

9.3. Index

```
@Indexed
private String categoryId;

@Indexed(unique = true)
private String sku;
```

Index không phải “bonus performance” sau cùng. Với MongoDB, query pattern và index là một phần của data model. Sinh viên phải nêu endpoint nào hưởng lợi từ index nào.

9.4. Verify index bằng Compass

26. Mở collection products trong Compass.
27. Mở tab Indexes.
28. Xác nhận _id_ index luôn có.
29. Sau khi app tạo index (nếu auto-index/config được bật theo course setup), kiểm tra index categoryId/sku.
30. Chụp screenshot làm evidence.

X. SEED DATA VÀ QUAN SÁT DOCUMENT THẬT

10.1. Seed Category trước Product

```
@Bean
CommandLineRunner seed(CategoryRepository categories,
                        ProductRepository products) {
    return args -> {
        if (categories.count() == 0) {
            Category accessories = categories.save(
                new Category(null, "Accessories")
            );

            products.save(new Product(
                null,
                "Mechanical Keyboard",
                new BigDecimal("120.00"),
                20,
                accessories.getId(),
                new ArrayList<>()
            ));
        }
    };
}
```

10.2. Expected document

```
{
  "_id": { "$oid": "..."},
  "name": "Mechanical Keyboard",
  "price": "120.00",
  "stock": 20,
  "categoryId": "...",
  "reviews": []
}
```

10.3. Compass evidence

31. Database: sba301_lab41.
32. Collection: categories.
33. Collection: products.
34. Mở một Product document.
35. Đối chiếu field trong Java với field BSON.
36. Thêm một embedded Review rồi quan sát document thay đổi.

XI. LAB WORKFLOW CHI TIẾT – 30 BƯỚC

1. Xác nhận JDK 21 và Maven.
2. Cài MongoDB Community Server.
3. Xác nhận Windows service MongoDB đang Running.
4. Cài/mở MongoDB Compass.
5. Connect mongodb://localhost:27017.
6. Tạo Spring Boot project với Web + MongoDB + Validation + Test.
7. Cấu hình spring.data.mongodb.uri.
8. Run app và kiểm tra không có connection error.
9. Tạo ProductRequest/ProductResponse DTO.
10. Tạo Category @Document.
11. Tạo Review embedded class.
12. Tạo Product @Document.
13. Tạo CategoryRepository.
14. Tạo ProductRepository.
15. Seed Category + Product.
16. Quan sát collections bằng Compass.
17. Tạo ProductService.
18. Implement category reference validation.
19. Implement findById/create/update/delete.
20. Tạo ProductController.
21. Test POST bằng Postman.
22. Test GET by ID.
23. Test PUT.
24. Test DELETE.
25. Thêm paging + sorting.
26. Thêm keyword/category filter.
27. Thêm @Indexed cho field query thường xuyên.
28. Thêm Review vào Product và quan sát embedded document.
29. Test validation/error cases.
30. Hoàn thiện README + evidence + comparison worksheet.

XII. POSTMAN VERIFICATION MATRIX

#	Request	Expected	Evidence
1	POST /api/products valid	201 + Location + body	Screenshot
2	POST invalid name	400	Error JSON
3	POST invalid categoryId	404/400 theo contract	Error JSON
4	GET existing id	200	Body
5	GET missing id	404	Error JSON
6	PUT existing id	200	Updated body
7	DELETE existing id	204	No body
8	GET page=0&size=5	200 + 5 items max	Page JSON
9	Sort by price	Correct order	Compare values
10	Filter category	Correct subset	Response
11	Create embedded Review	Product document changes	Compass
12	Duplicate unique sku	Controlled error	HTTP + log
13	Compass verify collections	products/categories exist	Screenshot
14	Refresh frontend/consumer	API contract unchanged	Optional

XIII. TESTING STRATEGY KHI ĐỔI DATABASE

13.1. Controller test – gần như không đổi

@WebMvcTest/MockMvc có thể mock ProductService; test controller contract không cần MongoDB thật.

13.2. Service unit test – gần như không đổi

Mockito mock ProductRepository/CategoryRepository. Mục tiêu là test business rules và mapping.

13.3. Repository/integration test – phải đổi

Không dùng @DataJpaTest. Với MongoDB, repository integration cần MongoDB test environment phù hợp. Có thể dùng local test database trong bài học, hoặc Testcontainers MongoDB nếu môi trường lớp hỗ trợ Docker.

13.4. Test pyramid comparison

Test level	JPA Lab	MongoDB Lab
Controller slice	@WebMvcTest + MockMvc	Giữ nguyên
Service unit	JUnit + Mockito	Giữ nguyên
Persistence slice/integration	@DataJpaTest/H2	MongoDB integration/Testcontainers/local test DB
End-to-end	REST → JPA → SQL DB	REST → MongoRepository → MongoDB

XIV. BẢNG SO SÁNH CHUYÊN SÂU JPA/SQL VS MONGODB

Khía cạnh	JPA / SQL	MongoDB	Câu hỏi cần trả lời
Model	Normalized tables	Documents/collections	Data nào được đọc cùng nhau?
Relationship	FK/JOIN	Embed/reference	Lifecycle và cardinality?
Schema	DDL constraints	Flexible document shape	Ài enforce validation?
Atomicity	Transaction across rows/tables	Single-document atomic; multi-doc possible	Có thể thiết kế aggregate trong 1 document?
Query	SQL/JPQL	Mongo query/aggregation	Query pattern chính là gì?
ID	Numeric/generated common	ObjectId/String common	API có phụ thuộc ID type không?
Migration	DDL migration important	Schema evolution/application logic	Document versioning cần không?
Index	B-tree etc.	Mongo indexes	Endpoint nào cần index?
Join heavy domain	Tự nhiên hơn	Có thể kém tự nhiên	Mongo có thực sự phù hợp domain?
Flexible events/logs	Có thể dùng JSON/related tables	Rất phù hợp nhiều use case	Dữ liệu thay đổi shape thường xuyên?

XV. BREAK-IT LAB VÀ TROUBLESHOOTING

15.1. Break-It scenarios

37. Stop MongoDB Windows service rồi gọi API. Ghi nhận lỗi.
38. Gửi categoryId không tồn tại. Kiểm tra business validation.
39. Gửi price âm. Kiểm tra @Valid.
40. Gửi JSON sai kiểu dữ liệu.
41. Sort bằng field không whitelist.
42. Thử embed 1.000 review giả và thảo luận vì sao design này xấu.
43. Xóa Category đang được Product reference; quan sát dangling reference nếu service không bảo vệ.
44. Tạo duplicate sku khi có unique index.

15.2. Debug decision tree

```

API fails
|
+-- App cannot start?
|   +-- Check MongoDB service / URI / port
|
+-- Request reaches controller?
|   +-- No: mapping / JSON / validation
|
+-- Service fails?
|   +-- business rule / missing reference
|
+-- Repository fails?
|   +-- query / index / mapping / Mongo connection
|
+-- Data "looks wrong"?
|   +-- inspect actual BSON document in Compass
  
```

XVI. BÀI TẬP TỔNG HỢP NÂNG CẤP LAB 04 → LAB 04.1

16.1. Upgrade task

Từ một REST API Product hiện đang dùng JPA/SQL trong Lab 04, tạo branch feature/lab41-mongodb và chuyển persistence sang MongoDB nhưng cố gắng giữ nguyên API contract.

16.2. Deliverable bắt buộc

Evidence	Mô tả
E01	Git branch + commit history
E02	Screenshot MongoDB service Running
E03	Compass connection localhost
E04	application.properties MongoDB URI
E05	Product/Category document classes
E06	Embed-vs-Reference Decision Sheet
E07	MongoRepository code
E08	Service validation cho manual reference
E09	REST Controller contract comparison
E10	Postman collection + verification matrix
E11	Compass product document + embedded review
E12	Index screenshot
E13	Automated test result
E14	JPA vs MongoDB comparison reflection
E15	README run instructions

16.3. Reflection bắt buộc

45. Những file nào giữ nguyên khi đổi database?
46. Những file nào thay đổi hoàn toàn?
47. Vì sao REST Controller không nên phụ thuộc trực tiếp MongoRepository?
48. Vì sao MongoDB không có nghĩa là “không cần schema/design”?
49. Nếu quay lại SQL Server, frontend có cần đổi không? Giải thích.

XVII. RUBRIC LAB 04.1 – 10 ĐIỂM

Tiêu chí	Điểm	Excellent	Đạt tối thiểu
MongoDB installation & configuration	0.75	Server + Compass + connection/evidence rõ	App kết nối được local MongoDB
Document model	1.25	Model rõ, validation, ID/field hợp lý	Product document chạy được
Embed/reference reasoning	1.50	Giải thích bằng access pattern/cardinality/lifecycle	Không map quan hệ máy móc
MongoRepository & queries	1.25	CRUD + paging/filter query rõ	CRUD repository hoạt động
Service & consistency	1.00	Business validation/reference consistency	Có service boundary
REST Controller/API contract	1.25	Status/DTO/error contract đúng	CRUD HTTP chạy
Paging/sorting/filter/index	0.75	Có guardrail + index reasoning	Có paging/sort
Testing & verification	0.75	Automated + Postman + Compass	Có verification evidence
Comparison & architecture explanation	1.00	Giải thích phần giữ/thay và trade-off	Trả lời được kiến trúc
Git/evidence/README	0.50	Evidence traceable, chạy lại được	Nộp đủ tối thiểu

Human verification rule: Code chạy là điều kiện cần. Sinh viên phải giải thích được ít nhất một quyết định embed/reference và một thay đổi architecture khi thay JPA bằng MongoDB.

XVIII. FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa trong Lab	Ví dụ
MongoDB	Document-oriented NoSQL database	products collection
Document	BSON record chứa fields/nested structures	Product document
Collection	Nhóm documents tương tự	products
BSON	Binary JSON representation MongoDB dùng	ObjectId/date types
ObjectId	Identifier thường dùng cho _id	Product _id
@Document	Spring Data mapping annotation	@Document(collection="products")
@Id	Mapping identifier	String id
MongoRepository	Spring Data repository abstraction cho MongoDB	ProductRepository
MongoTemplate	Lower-level Spring Data Mongo operations	Advanced dynamic query
Embedding	Lưu child data bên trong parent document	reviews trong Product
Reference	Lưu link/id tới document khác	categoryId
@DocumentReference	Spring Data document reference mapping	Category reference
@Indexed	Khai báo index mapping	categoryId/sku
Denormalization	Cố ý lặp dữ liệu để tối ưu access pattern	snapshot productName in OrderItem
Access pattern	Cách application thường đọc/ghi data	Product detail reads reviews
Atomicity	Update all-or-nothing boundary	single document update
Aggregation	Mongo processing pipeline	analytics/grouping
Compass	GUI quản trị/quan sát MongoDB	inspect products
mongosh	MongoDB shell	CLI query
URI	Connection string	mongodb://localhost:27017/sba301_lab41

XIX. AI-ASSISTED LEARNING & VERIFICATION

19.1. 15 prompt tự nghiên cứu

1. So sánh embedding và referencing trong MongoDB bằng Product–Review–Category.
2. Giải thích vì sao MongoDB không có nghĩa là schema-less application design.
3. Khi nào MongoDB document có nguy cơ tăng kích thước không kiểm soát?
4. So sánh JpaRepository và MongoRepository về abstraction và query model.
5. Giải thích single-document atomicity trong MongoDB.
6. Phân tích dangling reference khi lưu categoryId thủ công.
7. So sánh @DocumentReference và manual ID reference.
8. Đề xuất index cho endpoint search/filter Product và giải thích.
9. Khi nào relational JOIN-heavy domain phù hợp SQL hơn MongoDB?
10. Phân tích cách thiết kế Order document để giữ historical price snapshot.
11. Giải thích Testcontainers MongoDB dùng để làm gì.
12. Phân tích trade-off duplicate categoryName trong Product.
13. Đề xuất strategy migrate API backend từ JPA sang MongoDB mà frontend không đổi.
14. Giải thích aggregation pipeline ở mức beginner.
15. Đánh giá MongoDB có phù hợp cho hệ thống ngân hàng core ledger không và vì sao.

19.2. AI Verification Log

Prompt/AI claim	Nguồn kiểm chứng	Experiment/code	Kết luận của sinh viên

XX. SELF-CHECK & HUMAN VERIFICATION GATE

20.1. Self-check

Nội dung	Đạt	Chưa đạt	Evidence
Tôi cài và kết nối MongoDB local được.	☐	☐	
Tôi giải thích được collection/document/BSON.	☐	☐	
Tôi chuyển @Entity → @Document đúng.	☐	☐	
Tôi dùng MongoRepository với String ID.	☐	☐	
Tôi phân biệt embed và reference.	☐	☐	
Tôi biết MongoDB không enforce FK như SQL.	☐	☐	
Tôi giữ REST contract ổn định.	☐	☐	
Tôi thực hiện paging/sorting/filtering.	☐	☐	
Tôi giải thích index theo query pattern.	☐	☐	
Tôi biết controller/service test nào không cần MongoDB thật.	☐	☐	

20.2. Human Verification Gate – 15 câu

1. Vì sao Controller không inject MongoRepository trực tiếp?
2. Khi đổi JPA sang MongoDB, API contract nào nên giữ?
3. Vì sao Product.id đổi từ Long sang String nhưng frontend không nhất thiết phải quan tâm?
4. Phân biệt @Entity và @Document.
5. Tại sao không có @GeneratedValue giống JPA?
6. Vì sao Review có thể embed?
7. Vì sao Category có thể reference?
8. Nếu xóa Category, Product.categoryId sẽ thế nào?
9. Khi nào cần @Indexed?
10. Paging của MongoRepository có gì giống JpaRepository?
11. JPQL có dùng trực tiếp với MongoDB không?
12. @DataJpaTest có phù hợp repository MongoDB không?
13. Tại sao flexible schema vẫn cần DTO/validation?
14. Nêu một domain nên chọn SQL thay vì MongoDB.
15. Nêu một domain/data type MongoDB có lợi thế rõ ràng.

20.3. Exit ticket

- Một thứ em giữ nguyên từ Lab 04.
- Một thứ em buộc phải thay khi đổi database.
- Một quyết định embed/reference em có thể bảo vệ bằng lập luận.
- Một rủi ro nếu dùng MongoDB chỉ vì “NoSQL linh hoạt”.

PHỤ LỤC A – QUICK REFERENCE

JPA / SQL	MongoDB
@Entity	@Document
@Table	collection
@Id + @GeneratedValue	@Id / ObjectId/String
JpaRepository<T, Long>	MongoRepository<T, String>
FK	manual reference / @DocumentReference
@OneToMany child table	embedded list hoặc separate collection
JPQL	Mongo query / aggregation
@DataJpaTest	Mongo integration/slice approach
SQL client	MongoDB Compass

PHỤ LỤC B – EMBED/REFERENCE DECISION WORKSHEET

Quan hệ	Read together?	Shared?	Independent lifecycle?	Bounded?	Quyết định	Lý do

PHỤ LỤC C – JPA → MONGODB UPGRADE TRACE

Artifact	Lab 04 JPA	Lab 04.1 MongoDB	Keep/Change	Evidence
pom.xml				
application.properties				
Product model				
Relationship				
Repository				
Service				
Controller				
Exception handler				
Postman collection				
Persistence tests				

PHỤ LỤC D – README MINIMUM TEMPLATE

```
# SBA301 Lab 04.1 - REST + MongoDB

## Prerequisites
- JDK 21
- MongoDB Community Server
- MongoDB Compass

## Database
mongodb://localhost:27017/sba301_lab41

## Run
1. Start MongoDB service
2. Run Spring Boot application
3. Open Compass
4. Import/run Postman collection

## API
- GET /api/products
- GET /api/products/{id}
- POST /api/products
- PUT /api/products/{id}
- DELETE /api/products/{id}

## Modeling decision
Explain why Review is embedded and Category is referenced.

## Evidence
List E01-E15.
```

PHỤ LỤC E – COMPLETE IMPLEMENTATION PACK

E.1. CategoryRepository

```
public interface CategoryRepository
    extends MongoRepository<Category, String> {

    Optional<Category> findByNameIgnoreCase(String name);
}
```

E.2. ProductRepository với search/filter

```
public interface ProductRepository
    extends MongoRepository<Product, String> {

    Page<Product> findByNameContainingIgnoreCase(
        String keyword,
        Pageable pageable
    );

    Page<Product> findByCategoryId(
        String categoryId,
        Pageable pageable
    );

    Page<Product> findByNameContainingIgnoreCaseAndCategoryId(
        String keyword,
        String categoryId,
        Pageable pageable
    );
}
```

E.3. ProductService CRUD đầy đủ

```
@Service
public class ProductService {
    private static final Set<String> SORT_FIELDS =
        Set.of("name", "price", "stock");

    private final ProductRepository products;
    private final CategoryRepository categories;

    public ProductService(ProductRepository products,
        CategoryRepository categories) {
        this.products = products;
        this.categories = categories;
    }

    public ProductResponse findById(String id) {
        return toResponse(getRequired(id));
    }

    public ProductResponse create(ProductRequest request) {
        validateCategory(request.categoryId());
        Product p = new Product();
        apply(p, request);
        return toResponse(products.save(p));
    }

    public ProductResponse update(String id, ProductRequest request) {
        Product p = getRequired(id);
        validateCategory(request.categoryId());
        apply(p, request);
        return toResponse(products.save(p));
    }

    public void delete(String id) {
        Product p = getRequired(id);
    }
}
```

```

        products.delete(p);
    }

    private Product getRequired(String id) {
        return products.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Product not found: " + id));
    }

    private void validateCategory(String id) {
        if (id != null && !categories.existsById(id)) {
            throw new ResourceNotFoundException(
                "Category not found: " + id);
        }
    }

    private void apply(Product p, ProductRequest r) {
        p.setName(r.name());
        p.setPrice(r.price());
        p.setStock(r.stock());
        p.setCategoryId(r.categoryId());
    }
}

```

E.4. Search service với guardrail

```

public Page<ProductResponse> search(
    String keyword,
    String categoryId,
    int page,
    int size,
    String sortBy,
    String direction) {

    if (!SORT_FIELDS.contains(sortBy)) {
        throw new IllegalArgumentException("Unsupported sort field");
    }

    int safeSize = Math.min(Math.max(size, 1), 50);
    Sort.Direction dir = "desc".equalsIgnoreCase(direction)
        ? Sort.Direction.DESC : Sort.Direction.ASC;
    Pageable pageable = PageRequest.of(
        Math.max(page, 0), safeSize, Sort.by(dir, sortBy));

    Page<Product> result;
    if (!keyword.isBlank() && categoryId != null) {
        result = products.findByNameContainingIgnoreCaseAndCategoryId(
            keyword, categoryId, pageable);
    } else if (!keyword.isBlank()) {
        result = products.findByNameContainingIgnoreCase(keyword, pageable);
    } else if (categoryId != null) {
        result = products.findByCategoryId(categoryId, pageable);
    } else {
        result = products.findAll(pageable);
    }
    return result.map(this::toResponse);
}

```

E.5. Endpoint thêm embedded Review

```

public record ReviewRequest(
    @NotBlank String userId,
    @Min(1) @Max(5) int rating,
    @NotBlank String comment
) {}

public ProductResponse addReview(String productId,
    ReviewRequest request) {
    Product p = getRequired(productId);
}

```

```

        Review review = new Review(
            request.userId(),
            request.rating(),
            request.comment(),
            Instant.now()
        );
        p.getReviews().add(review);
        return toResponse(products.save(p));
    }

    @PostMapping("/{id}/reviews")
    public ResponseEntity<ProductResponse> addReview(
        @PathVariable String id,
        @Valid @RequestBody ReviewRequest request) {
        return ResponseEntity.ok(service.addReview(id, request));
    }

```

Sau request này, sinh viên phải mở Compass và chứng minh Review xuất hiện bên trong Product document, không phải trong một review table/collection riêng trong core lab.

PHỤ LỤC F – MONGODB COMPASS VERIFICATION WORKBOOK

F.1. Verify database và collections

50. Mở Compass và connect mongodb://localhost:27017.
51. Chọn database sba301_lab41.
52. Kiểm tra collections products và categories.
53. Chọn products → Documents.
54. Mở một document và xác nhận _id, name, price, stock, categoryId, reviews.

F.2. Thực hành filter trực tiếp trong Compass

```
{ "stock": { "$gt": 0 } }
{ "categoryId": "<replace-with-real-id>" }
{ "price": { "$gte": 50, "$lte": 200 } }
```

Mục tiêu không phải học Mongo shell syntax sâu, mà giúp sinh viên thấy repository query cuối cùng đang thao tác trên document fields thật.

F.3. Verify embedded structure

55. POST một review qua REST API.
56. Refresh Product document trong Compass.
57. Mở mảng reviews.
58. Xác nhận review nằm trong cùng Product document.
59. Thảo luận: nếu reviews tăng vô hạn thì data model cần thay đổi thế nào?

F.4. Verify manual reference

60. Copy categoryId từ Product.
61. Mở collection categories.
62. Tìm document có _id tương ứng.
63. Xác nhận MongoDB không tự hiển thị JOIN.
64. Xóa Category thử trong branch/seed test và quan sát Product vẫn có categoryId nếu Service không ngăn chặn.

F.5. Index verification

Index	Field	Use case	Expected evidence
id	_id	Lookup by id	Có mặc định
categoryId	categoryId	Filter category	Compass Indexes
sku unique	sku	Business uniqueness	Duplicate insert fails

PHỤ LỤC G – TESTING & VERIFICATION PACK

G.1. Service unit test example

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock ProductRepository products;
    @Mock CategoryRepository categories;
    @InjectMocks ProductService service;

    @Test
    void create_shouldRejectMissingCategory() {
        when(categories.existsById("missing")).thenReturn(false);

        ProductRequest request = new ProductRequest(
            "Keyboard", new BigDecimal("120"), 10, "missing");

        assertThatThrownBy(() -> service.create(request))
            .assertInstanceOf(ResourceNotFoundException.class);

        verify(products, never()).save(any());
    }
}
```

G.2. Controller contract test example

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean ProductService service;

    @Test
    void create_shouldReturn201() throws Exception {
        when(service.create(any())).thenReturn(
            new ProductResponse("p1", "Keyboard",
                new BigDecimal("120"), 10, "c1", List.of())
        );

        mvc.perform(post("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"name": "Keyboard", "price": 120,
                 "stock": 10, "categoryId": "c1"}
                """))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location", "/api/products/p1"));
    }
}
```

G.3. Persistence integration strategy

Core lab có thể test repository với MongoDB local test database để giảm yêu cầu môi trường. Nếu lớp đã có Docker, dùng Testcontainers MongoDB sẽ cho test repeatable và tránh phụ thuộc database của sinh viên.

Điểm kiểm tra	Unit/Mock	Mongo integration
Derived query naming	Không đủ	Cần
BSON mapping	Không đủ	Cần
Unique index behavior	Không đủ	Cần
Controller status	Đủ với MockMvc	Không cần DB
Business category validation	Đủ với Mockito	Có thể bổ sung

G.4. Minimum automated test set

65. Create valid product.

66. Reject invalid DTO.
67. Reject missing category.
68. Find existing product.
69. Return 404 for missing product.
70. Update existing product.
71. Delete existing product.
72. Search by keyword.
73. Filter by category.
74. Add embedded review.
75. Reject review rating ngoài 1–5.
76. Verify unique sku/index nếu triển khai.

PHỤ LỤC H – ARCHITECTURE CHANGE REVIEW

H.1. File-by-file change map

File/Artifact	Giữ nguyên?	Thay đổi gì?	Lý do
ProductController	Gần như	Long id → String id nếu API lộ id type	Database identifier model
DTOs	Gần như	Có thể thêm Mongo-specific constraints? Nên tránh	API contract độc lập DB
GlobalExceptionHandler	Có	Có thể thêm duplicate-key mapping	Error contract ổn định
ProductService	Phần lớn	Reference consistency/query semantics	Mongo không enforce FK
ProductRepository	Không	JpaRepository → MongoRepository	Persistence technology
Product entity/model	Không	@Entity → @Document; relations → embed/reference	Data model
application.properties	Không	datasource/JPA → mongodb URI	Connection/config
Repository tests	Không	@DataJpaTest → Mongo integration	Persistence engine
React frontend	Có	Không cần đổi nếu API contract tương thích	Layer separation

H.2. Kiến trúc cần sinh viên vẽ lại

Before:
 React -> REST Controller -> Service -> JpaRepository -> SQL Server

After:
 React -> REST Controller -> Service -> MongoRepository -> MongoDB

Stable boundary:

```

[ HTTP API contract ]
    |
[ application logic ]
    |
[ persistence adapter ] <-- replaced
  
```

H.3. Câu hỏi phản biện

77. Nếu Controller import MongoRepository trực tiếp, việc đổi database còn “replaceable” không?
78. Nếu API trả ObjectId/BSON-specific object thay vì String DTO, frontend bị coupling thế nào?
79. Nếu MongoDB phù hợp cho logs nhưng domain Order/Payment cần relational consistency mạnh, có nhất thiết chuyển toàn bộ hệ thống sang MongoDB không?
80. Có thể dùng SQL Server và MongoDB cùng lúc trong một project không? Hãy nêu data nào phù hợp mỗi bên.